IF2211 Strategi Algoritma
Laporan Tugas Kecil 2
Voxelization Objek 3D menggunakan Octree

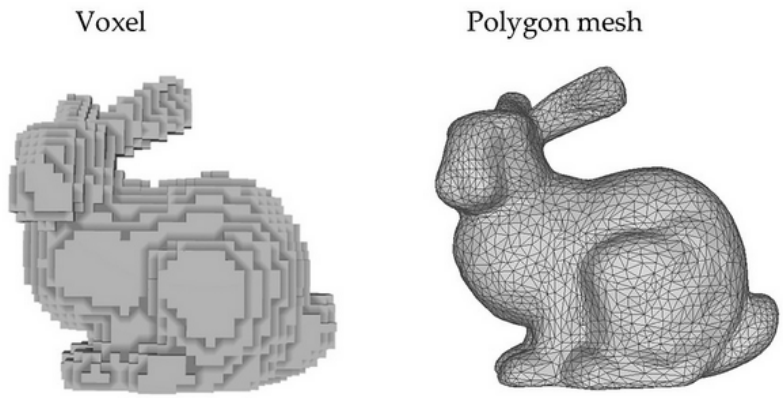
Disusun oleh:
Safira Berlianti (13524128)

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

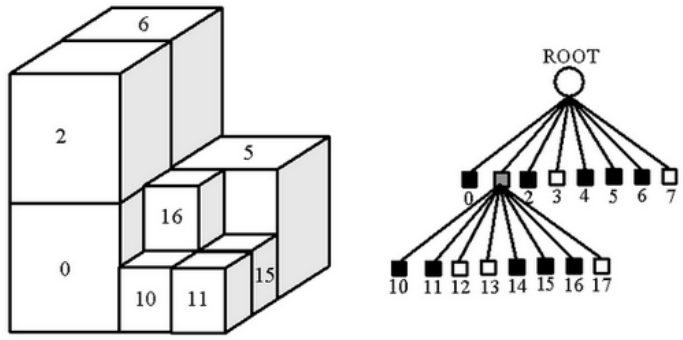
BAB 1 DESKRIPSI TUGAS **3**
BAB 2 DESAIN STRATEGI DIVIDE AND CONQUER **5**
2.1. Konsep Divide and Conquer yang Digunakan 5
2.2. Langkah-Langkah Algoritma 5
BAB 3 IMPLEMENTASI PROGRAM **7**
3.1. struct.hpp 7
3.2. octree.cpp 8
3.2.1. Prosedur subdivide 8
3.2.2. Fungsi intersect 9
3.2.3. Fungsi buildOctree 9
3.3. io.cpp 11
3.3.1. Fungsi parseOBJ 11
3.3.2. Prosedur exportToOBJ 13
3.3.3. Prosedur generateCubeVertices 15
3.3.4. Prosedur generateCubeFaces 15
3.4. main.cpp 16
BAB 4 PENGUJIAN **19**
4.1. Testcase 1 19
4.2. Testcase 2 19
4.3. Testcase 3 20
4.4. Testcase 4 21
4.5. Testcase 5 22
BAB 5 KESIMPULAN **23**
LAMPIRAN **24**

BAB 1 DESKRIPSI TUGAS



Gambar 1 Visualisasi *voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

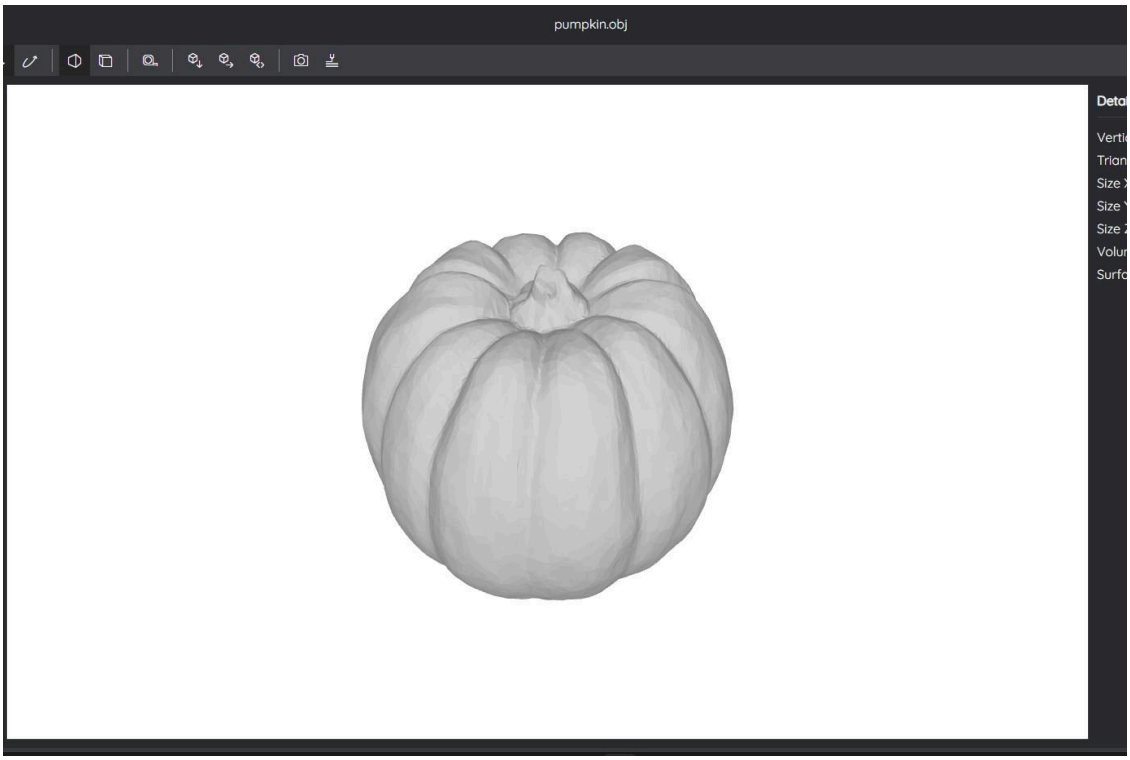


Gambar 2 Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

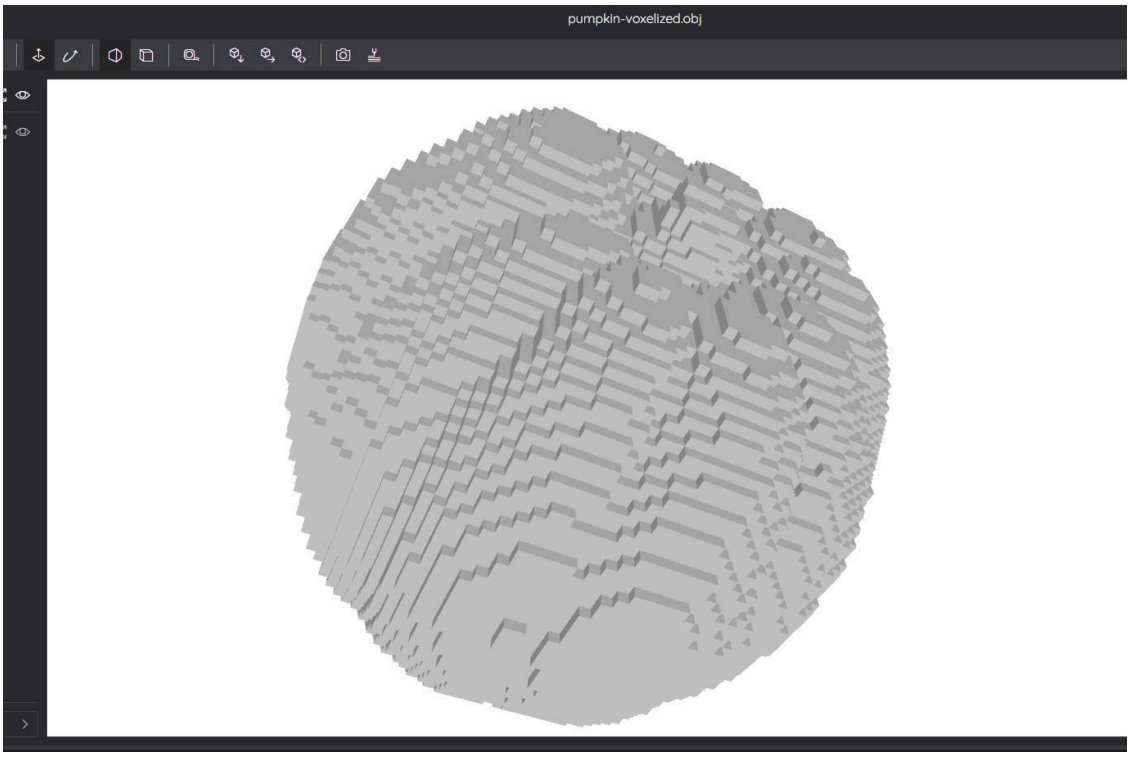
Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari *voxel* dengan memanfaatkan properti ***Octree***. Program melakukan konversi model dengan menggunakan algoritma ***divide and conquer*.**

Ilustrasi Kasus

Diberikan sebuah model 3D sebagai berikut.



Di bawah adalah hasil perubahan.



BAB 2 DESAIN STRATEGI DIVIDE AND CONQUER

2.1. Konsep Divide and Conquer yang Digunakan

Algoritma yang digunakan mengikuti paradigma Divide and Conquer, yang terdiri dari tiga tahap utama:

1. **Divide**
   Ruang 3D yang berisi objek pertama-tama direpresentasikan sebagai sebuah bounding box berbentuk kubus. Kemudian, bounding box ini dibagi menjadi 8 bagian sama besar, setiap bagian merupakan sub-masalah yang lebih kecil. Proses ini dilakukan secara rekursif hingga mencapai kedalaman maksimum yang ditentukan.

2. **Conquer**
   Untuk setiap sub-kubus hasil pembagian dicek apakah kubus tersebut beririsan dengan objek. Hal ini ditentukan dengan mengecek apakah ada segitiga (face) yang berada di dalam atau memotong kubus tersebut. Jika tidak ada segitiga yang beririsan, maka kubus diabaikan (pruning). Namun jika ada, maka kubus diproses lebih lanjut. Jika sudah mencapai depth maksimum, kubus tersebut dianggap sebagai voxel dan disimpan sebagai hasil akhir

3. **Combine**
   Semua voxel yang dihasilkan dari leaf node pada depth maksimum dikumpulkan menjadi satu kesatuan dan digunakan untuk membentuk model akhir dalam format .obj.

2.2. Langkah-Langkah Algoritma

1. File .obj dibaca baris per baris. Kemudian, vertex disimpan dalam array, face dibentuk dari kombinasi 3 vertex, dan dihitung bounding box minimum dan maksimum. Bounding box kemudian diubah menjadi kubus.

2. Root node dibuat dengan bounding box awal, depth dimulai dari 0.

3. Setiap node dibagi menjadi 8 child node. Pembagian dilakukan dengan mencari titik tengah. Akan terbentuk 8 bounding box baru.

4. Untuk setiap child dicek apakah face berpotongan dengan bounding box. Jika tidak ada face yang berpotongan, maka node tidak ditelusuri. Jika ada face, maka rekursi akan dilanjutkan.

5. Proses divide diulang pada child node. Depth bertambah setiap level.

6. Jika depth sudah mencapai maksimum, maka node menjadi leaf, serta dibentuk voxel dengan center yang merupakan titik tengah bounding box dan size yang merupakan panjang sisi kubus. Kemudian, voxel disimpan.

7. Setiap voxel diubah menjadi 8 vertex dan 12 face. Semua voxel digabungkan menjadi satu file .obj.

BAB 3 IMPLEMENTASI PROGRAM

3.1. struct.hpp

```
structure Vertex
   x, y, z : float
end structure

structure Face
   v1, v2, v3 : Vertex
end structure

structure BoundingBox
   minB, maxB : Vertex
end structure

structure Voxel
   center : Vertex
   size : float
end structure

structure OctreeNode
   box : BoundingBox
   depth : integer
   isLeaf : boolean
   children : array[0..7] of OctreeNode pointer
end structure
```

Berikut adalah penjelasan struct yang digunakan:

1. Vertex
   Struct ini merepresentasikan koordinat titik dalam sistem koordinat cartesian 3 dimensi. Struct ini memiliki tiga atribut, yaitu x (koordinat pada sumbu X), y (koordinat pada sumbu Y), dan z (koordinat pada sumbu Z).

2. Face
   Struct ini merepresentasikan face dan menyimpan tiga buah vertex yang membentuk sebuah segitiga.

3. BoundingBox
   Struct ini merepresentasikan kotak pembatas yang digunakan untuk partisi ruang. Struct ini menyimpan dua vertex, yaitu minB (vertex dengan koordinat minimum) dan maxB (vertex dengan koordinat maksimum).

4. ParsedData

Struct ini menampung hasil parsing file OBJ yang sudah diproses. Struct ini memiliki dua atribut, yaitu faces (vektor yang berisi semua face segitiga dari model OBJ) dan bound (bounding box kubus objek yang belum dibagi).

5. Voxel
   Struct ini merepresentasikan sebuah voxel dalam ruang 3D.Struct ini memiliki dua atribut, yaitu center (vertex titik pusat dari voxel) dan size (panjang sisi voxel). Voxel dihasilkan dari leaf octree pada kedalaman maksimum.

6. OctreeNode
   Struct ini merepresentasikan sebuah node dalam struktur data octree. Struct ini memiliki empat atribut:
   - box: bounding box yang mendefinisikan wilayah ruang yang dicakup oleh node
   - depth: tingkat kedalaman node dalam pohon
   - isLeaf: boolean yang menandakan apakah suatu node adalah leaf
   - children: array berisi 8 pointer ke node anak (nullptr jika node adalah leaf)

3.2. octree.cpp

3.2.1. Prosedur subdivide

```
procedure subdivide(input parent : BoundingBox, output child : array[0..7] of BoundingBox)
{ Membagi bounding box parent menjadi 8 anak kotak yang sama besar }

Kamus
   mid : Vertex
   xs, ys, zs : array[0..2] of float
   i, j, k, idx : integer

Algoritma:
   mid.x ← (parent.minB.x + parent.maxB.x) / 2
   mid.y ← (parent.minB.y + parent.maxB.y) / 2
   mid.z ← (parent.minB.z + parent.maxB.z) / 2

   xs ← [parent.minB.x, mid.x, parent.maxB.x]
   ys ← [parent.minB.y, mid.y, parent.maxB.y]
   zs ← [parent.minB.z, mid.z, parent.maxB.z]

   idx ← 0
   for i ← 0 to 1 do
      for j ← 0 to 1 do
         for k ← 0 to 1 do
            child[idx].minB ← {xs[i], ys[j], zs[k]}
            child[idx].maxB ← {xs[i+1], ys[j+1], zs[k+1]}
```

```
            idx ← idx + 1
        endfor
    endfor
endfor
```

Prossedur ini berfungsi untuk membagi sebuah bounding box menjadi 8 sub-bounding box yang sama besar. Hal ini dilakukan dengan menghitung titik tengah (mid) dari bounding box parent. Kemudian, dilakukan iterasi untuk menghasilkan 8 kombinasi bounding box. Setiap child adalah bounding box yang merupakan 1/8 dari volume parent.

3.2.2. Fungsi intersect

```
function intersect(b : BoundingBox, f : Face) → boolean
{ Memeriksa apakah bounding box b beririsan dengan segitiga f }

Kamus
   tMinX, tMaxX, tMinY, tMaxY, tMinZ, tMaxZ : float

Algoritma:
   tMinX ← min(f.v1.x, f.v2.x, f.v3.x)
   tMaxX ← max(f.v1.x, f.v2.x, f.v3.x)
   tMinY ← min(f.v1.y, f.v2.y, f.v3.y)
   tMaxY ← max(f.v1.y, f.v2.y, f.v3.y)
   tMinZ ← min(f.v1.z, f.v2.z, f.v3.z)
   tMaxZ ← max(f.v1.z, f.v2.z, f.v3.z)

   return (b.minB.x ≤ tMaxX) and (b.maxB.x ≥ tMinX) and
          (b.minB.y ≤ tMaxY) and (b.maxB.y ≥ tMinY) and
          (b.minB.z ≤ tMaxZ) and (b.maxB.z ≥ tMinZ)
```

Fungsi ini berfungsi untuk memeriksa apakah sebuah bounding box beririsan dengan sebuah face segitiga. Hal ini dilakukan dengan cara menghitung bounding box dari segitiga dengan mencari nilai minimum dan maksimum dari ketiga vertex pada sumbu x, y, z. Dua bounding box (kotak dari octree dan kotak dari segitiga) beririsan jika dan hanya jika kotak octree min X ≤ kotak segitiga max X dan kotak octree max X ≥ kotak segitiga min X, hal ini juga berlaku untuk sumbu Y dan Z. Jika semua kondisi terpenuhi, fungsi akan mengembalikan true.

3.2.3. Fungsi buildOctree

```
procedure buildOctree(node : OctreeNode pointer, faces : vector of Face,
                      curDepth : integer, maxDepth : integer,
                      nodesCreated : vector of integer,
                      nodesSkipped : vector of integer,
                      voxels : vector of Voxel)
```

```
{ Membangun octree secara rekursif dengan algoritma divide and conquer }

Kamus
   i : integer
   childBox : array[0..7] of BoundingBox
   relevant : vector of Face
   childNode : OctreeNode pointer

Algoritma:
   nodesCreated[curDepth] ← nodesCreated[curDepth] + 1

   if curDepth = maxDepth then
      node.isLeaf ← true

      Voxel vox
      vox.center.x ← (node.box.minB.x + node.box.maxB.x) / 2
      vox.center.y ← (node.box.minB.y + node.box.maxB.y) / 2
      vox.center.z ← (node.box.minB.z + node.box.maxB.z) / 2
      vox.size ← node.box.maxB.x - node.box.minB.x
      voxels.push_back(vox)

      return
   else
      node.isLeaf ← false
   endif

   for i ← 0 to 7 do
      node.children[i] ← nullptr
   endfor

   subdivide(node.box, childBox)

   for i ← 0 to 7 do
      relevant.clear()

      for each face f in faces do
         if intersect(childBox[i], f) then
            relevant.push_back(f)
         endif
      endfor

      if relevant.empty() then
         nodesSkipped[curDepth + 1] ← nodesSkipped[curDepth + 1] + 1
         continue
      endif
```

```
    childNode ← new OctreeNode
    childNode.box ← childBox[i]
    childNode.depth ← curDepth + 1
    childNode.isLeaf ← false
    node.children[i] ← childNode

    buildOctree(childNode, relevant, curDepth + 1, maxDepth,
            nodesCreated, nodesSkipped, voxels)
  endfor
```

Prosedur ini berfungsi membangun octree. Prosedur ini menerima node octree, daftar face, kedalaman saat ini, dan kedalaman maksimum. Prosedur ini menghasilkan jumlah node yang ditelusuri, node yang di-skip, dan kumpulan voxel hasil.

- Base case: Jika kedalaman saat ini sama dengan kedalaman maksimum, maka node ditandai sebagai leaf, membuat voxel dari bounding box node dengan pusat di tengah kotak dan ukuran panjang sisi, dan menyimpan voxel ke dalam vektor hasil.

- Rekursif: Jika belum mencapai kedalaman maksimum, maka node ditandai bukan leaf dan dibuat 8 node children. Kemudian dipanggil subdivide untuk mendapatkan 8 child bounding box. Untuk setiap child box dicari face-face yang beririsan dengan child box (menggunakan fungsi intersect). Jika tidak ada face yang beririsan, child box di-skip (tidak dibuat node-nya) dan statistik nodesSkipped ditambah. Jika ada face yang beririsan akan dibuat node child baru dan dipanggil buildOctree secara rekursif dengan face yang relevan dan statistik nodesCreated untuk node yang dibuat akan ditambah.

3.3. io.cpp

3.3.1. Fungsi parseOBJ

```
function parseOBJ(path : string) → ParsedData
{ Membaca file OBJ dan mengembalikan data terstruktur yang berisi faces dan bounding box }

Kamus
   file : ifstream
   res : ParsedData
   vertices : vector of Vertex
   faces : vector of Face
   box : BoundingBox
   line : string
   ss : stringstream
   type : string
   v : Vertex
```

```
    a, b, c : integer
    maxSide : float
    center : Vertex
    halfSide : float

Algoritma:
    buka file dengan path

    if file tidak terbuka then
        cetak "Failed to open file"
        exit(1)
    endif

    { Inisialisasi bounding box dengan nilai ekstrim }
    box.minB ← {3.40282347e+38, 3.40282347e+38, 3.40282347e+38}
    box.maxB ← {-3.40282347e+38, -3.40282347e+38, -3.40282347e+38}

    while (baca baris dari file) do
        if baris kosong then
            continue
        endif

        ss ← stringstream(line)
        ss >> type

        if type = "v" then { Vertex }
            if ss >> v.x >> v.y >> v.z then
                vertices.push_back(v)

                { Update bounding box min }
                if v.x < box.minB.x then box.minB.x ← v.x endif
                if v.y < box.minB.y then box.minB.y ← v.y endif
                if v.z < box.minB.z then box.minB.z ← v.z endif

                { Update bounding box max }
                if v.x > box.maxB.x then box.maxB.x ← v.x endif
                if v.y > box.maxB.y then box.maxB.y ← v.y endif
                if v.z > box.maxB.z then box.maxB.z ← v.z endif
            endif

        else if type = "f" then { Face }
            if ss >> a >> b >> c then
                if (a > 0 and a ≤ vertices.size()) and
                  (b > 0 and b ≤ vertices.size()) and
                  (c > 0 and c ≤ vertices.size()) then
```

```
            Face f ← {vertices[a-1], vertices[b-1], vertices[c-1]}
            faces.push_back(f)
          endif
        endif
      endif
    endwhile

    { Buat bounding box kubus seragam }
    maxSide ← max(box.maxB.x - box.minB.x,
              box.maxB.y - box.minB.y,
              box.maxB.z - box.minB.z)

    center.x ← (box.minB.x + box.maxB.x) / 2
    center.y ← (box.minB.y + box.maxB.y) / 2
    center.z ← (box.minB.z + box.maxB.z) / 2

    halfSide ← maxSide / 2

    box.minB ← {center.x - halfSide, center.y - halfSide, center.z - halfSide}
    box.maxB ← {center.x + halfSide, center.y + halfSide, center.z + halfSide}

    res.faces ← faces
    res.bound ← box

    return res
```

Fungsi ini membaca file OBJ dan mengambil informasi geometri dari file tersebut. Fungsi ini membuka file OBJ, membaca file baris per baris, dan memproses dua jenis data, yaitu vertex dan face. Setiap face disimpan dalam vektor faces. Setiap vertex disimpan dalam vektor vertices dan digunakan untuk memperbarui bounding box yang mencakup seluruh objek. Setelah semua data dibaca, bounding box diubah menjadi bentuk kubus (sisi sama panjang) dengan mencari sisi terpanjang dari bounding box asli, lalu membuat kubus dengan pusat yang sama. Fungsi ini mengembalikan struktur ParsedData yang berisi daftar face dan bounding box yang sudah diseragamkan

3.3.2. Prosedur exportToOBJ

```
procedure exportToObj(filename : string, voxels : vector of Voxel)
{ Mengekspor voxel ke file OBJ dengan vertex dan face kubus }

Kamus
  outFile : ofstream
  startIdx : integer
  vox : Voxel
```

```
    v : array[0..7] of Vertex
    faces : array[0..11][0..2] of integer
    i : integer

Algoritma:
    outFile ← ofstream(filename)

    if outFile tidak terbuka then
        cetak "Gagal membuka file: " + filename
        return
    endif

    startIdx ← 1

    for each vox in voxels do
        generateCubeVertices(vox, v)

        { Tulis 8 vertex }
        for i ← 0 to 7 do
            outFile << "v " << v[i].x << " " << v[i].y << " " << v[i].z << "\n"
        endfor

        generateCubeFaces(startIdx, faces)

        { Tulis 12 face segitiga }
        for i ← 0 to 11 do
            outFile << "f " << faces[i][0] << " " << faces[i][1] << " " << faces[i][2] << "\n"
        endfor

        startIdx ← startIdx + 8
    endfor

    close outFile
    output ("File ", filename, " berhasil ditulis.")
```

Prosedur ini mengekspor voxel-voxel yang dihasilkan ke dalam file OBJ. Hal ini dilakukan dengan membuka file output untuk ditulis. Kemudian, untuk setiap voxel dalam daftar voxel dipanggil generateCubeVertices untuk menghasilkan 8 vertex kubus berdasarkan pusat dan ukuran voxel, 8 vertex tersebut ditulis ke file dengan format v x y z. Kemudian, dipanggil generateCubeFaces untuk menghasilkan 12 face segitiga, 12 face tersebut ditulis dengan format f i1 i2 i3. Setiap voxel menambahkan 8 vertex baru ke file, sehingga indeks vertex untuk voxel berikutnya perlu ditambahkan offset.

3.3.3. Prosedur generateCubeVertices

```
procedure generateCubeVertices(input voxel : Voxel, output out : array[0..7] of Vertex)
{ Menghasilkan 8 vertex dari sebuah voxel }

Kamus
   h : float

Algoritma:
   h ← voxel.size / 2

   out[0] ← {voxel.center.x - h, voxel.center.y - h, voxel.center.z - h}
   out[1] ← {voxel.center.x + h, voxel.center.y - h, voxel.center.z - h}
   out[2] ← {voxel.center.x + h, voxel.center.y + h, voxel.center.z - h}
   out[3] ← {voxel.center.x - h, voxel.center.y + h, voxel.center.z - h}
   out[4] ← {voxel.center.x - h, voxel.center.y - h, voxel.center.z + h}
   out[5] ← {voxel.center.x + h, voxel.center.y - h, voxel.center.z + h}
   out[6] ← {voxel.center.x + h, voxel.center.y + h, voxel.center.z + h}
   out[7] ← {voxel.center.x - h, voxel.center.y + h, voxel.center.z + h}
```

Prosedur ini menerima sebuah voxel dan menghasilkan 8 vertex sudut dari voxel tersebut. Hal ini dilakukan dengan menghitung h = size/2, yaitu setengah panjang sisi voxel. Kemudian, menghasilkan 8 vertex dengan koordinat berdasarkan kombinasi x dan y dengan nilai ±h dari pusat.

3.3.4. Prosedur generateCubeFaces

```
procedure generateCubeFaces(input startIdx : integer, output faces : array[0..11][0..2] of
integer)
{ Menghasilkan indeks face untuk kubus berdasarkan pola tetap }

Kamus
   pattern : constant array[0..11][0..2] of integer
   i : integer

Algoritma:
   { Pola face kubus dalam format segitiga }
   pattern ← {
      {0, 2, 1}, {0, 3, 2},  { Face depan (z negatif) }
      {4, 5, 6}, {4, 6, 7},  { Face belakang (z positif) }
      {0, 1, 5}, {0, 5, 4},  { Face bawah (y negatif) }
      {3, 6, 2}, {3, 7, 6},  { Face atas (y positif) }
      {0, 7, 3}, {0, 4, 7},  { Face kiri (x negatif) }
      {1, 2, 6}, {1, 6, 5}   { Face kanan (x positif) }
   }
```

```
for i ← 0 to 11 do
    faces[i][0] ← startIdx + pattern[i][0]
    faces[i][1] ← startIdx + pattern[i][1]
    faces[i][2] ← startIdx + pattern[i][2]
endfor
```

Prosedur ini menghasilkan indeks-indeks vertex untuk membentuk 12 face segitiga dari sebuah kubus. Prosedur ini menggunakan pola tetap pattern yang telah didefinisikan sebagai array 12×3. Setiap baris dalam pattern berisi tiga indeks yang mereferensikan vertex dari array 8 vertex yang dihasilkan oleh generateCubeVertices. Pola ini mengatur:

- Face depan (z negatif): 2 segitiga dari vertex 0,1,2,3
- Face belakang (z positif): 2 segitiga dari vertex 4,5,6,7
- Face bawah (y negatif): 2 segitiga dari vertex 0,1,5,4
- Face atas (y positif): 2 segitiga dari vertex 3,6,2,7
- Face kiri (x negatif): 2 segitiga dari vertex 0,7,3,4
- Face kanan (x positif): 2 segitiga dari vertex 1,2,6,5

Indeks yang dihasilkan ditambahkan dengan startIdx untuk menyesuaikan dengan posisi vertex dalam file OBJ.

3.4. main.cpp

```
Program main()
{ Program utama untuk membangun octree dari file OBJ dan mengekspor voxel }

Kamus
    pathIn, pathOut : string
    maxDepth : integer
    start, stop : time_point
    duration : duration
    data : ParsedData
    nodesCreated : vector of integer
    nodesSkipped : vector of integer
    voxels : vector of Voxel
    root : OctreeNode pointer
    i : integer

Algoritma:
    { Input dari pengguna }
    output ("Path file .obj: ")
    input (pathIn)
```

```
output ("Nama file output (.obj): ")
input (pathOut)

output ("Input max depth: ")
input (maxDepth)

{ Mulai timer }
start ← high_resolution_clock::now()

{ Parsing file OBJ }
data ← parseOBJ(pathIn)

if data.faces.empty() then
   output ("Gagal membaca file atau model tidak valid.")
endif

{ Inisialisasi statistik }
nodesCreated ← vector of integer dengan ukuran maxDepth+1, nilai awal 0
nodesSkipped ← vector of integer dengan ukuran maxDepth+1, nilai awal 0
voxels ← empty vector of Voxel

{ Inisialisasi root octree }
root ← new OctreeNode
root.box ← data.bound
root.depth ← 0

{ Bangun octree }
buildOctree(root, data.faces, 0, maxDepth, nodesCreated, nodesSkipped, voxels)

{ Hentikan timer }
stop ← high_resolution_clock::now()
duration ← duration_cast<milliseconds>(stop - start)

{ Ekspor hasil }
if not voxels.empty() then
   exportToObj(pathOut, voxels)
   output ("File ", pathOut, " berhasil dibuat dengan ", voxels.size(), " titik.")
else
   output ("Voxel kosong, tidak ada yang diekspor.")
endif

{ Cetak statistik }
output ("\n================ STATISTIK ================")
output ("Banyaknya voxel yang terbentuk: ", voxels.size())
output ("Banyaknya vertex yang terbentuk: ", (voxels.size() * 8))
```

```
output ("Banyaknya faces yang terbentuk: ", (voxels.size() * 12))

output ("\nStatistik node octree yang terbentuk:")
for i ← 1 to maxDepth do
   output (i, " : ", nodesCreated[i])
endfor

output ("\nStatistik node yang tidak perlu ditelusuri:")
for i ← 1 to maxDepth do
   output (i, " : ", nodesSkipped[i])
endfor

output ("\nKedalaman octree: ", maxDepth)
output ("Lama waktu program berjalan: ", duration.count(), " ms")
output ("Path file .obj: ", pathOut)
output ("==========================================")

return 0
```

Program ini merupakan program utama pada tugas kecil ini. Program ini melakukan:

1. Input**:** Meminta pengguna memasukkan path file OBJ input, nama file output, dan kedalaman maksimum octree.

2. Timer**:** Memulai pencatatan waktu eksekusi.

3. Parsing**:** Memanggil parseOBJ untuk membaca file OBJ.

4. Inisialisasi**:** Membuat vektor untuk statistik nodesCreated dan nodesSkipped, vektor voxels untuk menyimpan hasil, dan node root octree dengan bounding box dari objek.

5. Pembangunan Octree**:** Memanggil buildOctree untuk membangun struktur octree.

6. Ekspor**:** Jika voxel tidak kosong, memanggil exportToObj untuk menulis hasil ke file.

7. Statistik**:** Menampilkan informasi jumlah voxel, vertex, dan face yang dihasilkan, statistik node octree per kedalaman, statistik node yang di-skip per kedalaman, dan waktu eksekusi program.

BAB 4 PENGUJIAN

4.1. Testcase 1

Input
Path file .obj: test/line.obj
Nama file output (.obj): test/out_line.obj
Input max depth: 2

Output
========== STATISTIK ==========
Banyaknya voxel yang terbentuk: 32
Banyaknya vertex yang terbentuk: 256
Banyaknya faces yang terbentuk: 384

Statistik node octree yang terbentuk:
1 : 8
2 : 32

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 32

Kedalaman octree: 2
Lama waktu program berjalan: 5 ms
Path file .obj: test/out_line.obj
==========
{ file output dapat dilihat di repository }

4.2. Testcase 2

Input
Path file .obj: test/teapot.obj
Nama file output (.obj): test/out_teapot.obj
Input max depth: 6

Output
========== STATISTIK ==========
Banyaknya voxel yang terbentuk: 17892
Banyaknya vertex yang terbentuk: 143136
Banyaknya faces yang terbentuk: 214704

Statistik node octree yang terbentuk:
1 : 8
2 : 26
3 : 128
4 : 573
5 : 3044
6 : 17892

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 38
3 : 80
4 : 451
5 : 1540
6 : 6460

Kedalaman octree: 6
Lama waktu program berjalan: 90 ms
Path file .obj: test/out_teapot.obj
==========
{ file output dapat dilihat di repository }

4.3. Testcase 3

Input
Path file .obj: test/pumpkin.obj
Nama file output (.obj): test/out_pumpkin.obj
Input max depth: 8

Output
========== STATISTIK ==========
Banyaknya voxel yang terbentuk: 1224161
Banyaknya vertex yang terbentuk: 9793288
Banyaknya faces yang terbentuk: 14689932

Statistik node octree yang terbentuk:
1 : 8
2 : 56
3 : 300
4 : 1345
5 : 6421

6 : 33134
7 : 190455
8 : 1224161

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 148
4 : 1055
5 : 4339
6 : 18234
7 : 74617
8 : 299479

Kedalaman octree: 8
Lama waktu program berjalan: 1676 ms
Path file .obj: test/out_pumpkin.obj
==========
{ file output tidak dapat dilihat di repository karena sizenya terlalu besar }

4.4. Testcase 4

Input
Path file .obj: test/cow.obj
Nama file output (.obj): test/out_cow.obj
Input max depth: 7

Output
========== STATISTIK ==========
Banyaknya voxel yang terbentuk: 49071
Banyaknya vertex yang terbentuk: 392568
Banyaknya faces yang terbentuk: 588852

Statistik node octree yang terbentuk:
1 : 8
2 : 20
3 : 96
4 : 372
5 : 1764
6 : 8797
7 : 49071

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 44
3 : 64
4 : 396
5 : 1212
6 : 5315
7 : 21305

Kedalaman octree: 7
Lama waktu program berjalan: 228 ms
Path file .obj: test/out_cow.obj
==========
{ file output dapat dilihat di repository }

4.5. Testcase 5

Input
Path file .obj: test/pumpkin.obj
Nama file output (.obj): test/test5_pumpkin.obj
Input max depth: 4

Output
========== STATISTIK ==========
Banyaknya voxel yang terbentuk: 1345
Banyaknya vertex yang terbentuk: 10760
Banyaknya faces yang terbentuk: 16140

Statistik node octree yang terbentuk:
1 : 8
2 : 56
3 : 300
4 : 1345

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 148
4 : 1055

Kedalaman octree: 4
Lama waktu program berjalan: 172 ms
Path file .obj: test/test5_pumpkin.obj
==========
{ file output dapat dilihat di repository }

BAB 5 KESIMPULAN

Pada tugas ini, telah berhasil diimplementasikan proses voxelization objek 3D dari file .obj menggunakan pendekatan Divide and Conquer dengan struktur data Octree. Algoritma bekerja dengan cara membagi ruang 3D secara rekursif menjadi delapan bagian hingga mencapai kedalaman tertentu. Pada setiap pembagian, hanya bagian ruang yang beririsan dengan permukaan objek yang diproses lebih lanjut, sedangkan bagian lain diabaikan. Pendekatan ini membuat proses menjadi lebih efisien karena menghindari perhitungan pada area kosong.Hasil akhir berupa kumpulan voxel berbentuk kubus uniform yang merepresentasikan permukaan objek, yang kemudian dikonversi kembali menjadi file .obj dalam bentuk mesh segitiga.

Dari implementasi ini dapat disimpulkan bahwa:

- Pendekatan Divide and Conquer dengan Octree efektif untuk membagi ruang dan mengurangi kompleksitas perhitungan.
- Proses pruning (mengabaikan node tanpa interaksi) sangat berpengaruh terhadap efisiensi.
- Tingkat kedalaman (depth) menentukan tingkat detail voxel: semakin tinggi depth, semakin halus hasilnya namun dengan biaya komputasi lebih besar.
- Representasi voxel memungkinkan visualisasi objek dalam bentuk diskret yang seragam.

LAMPIRAN

Pranala repository

https://github.com/Hwiyeahh/Tucil2_13524128

Pernyataan tidak melakukan kecurangan yang ditandatangani

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Safira Berlianti